P13A5ML2 MACHINE LEARNING-II

VAGHELA SUMIT H.

MCA SEM 3 ENROLLMENT NO. 25034211082

1. Write Python code for binary logistic regression with default parameters using Heart.csv dataset.

```python
# Binary Logistic Regression using Heart.csv

# Step 1: Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Load the dataset
data = pd.read_csv("D:\\A_AMPICS_MCA\\sem_3\\ml\\dataset\\heart.csv")

# Step 3: Display first 5 records
print("First 5 records:")
print(data.head())

# Step 4: Separate input features (X) and target variable (y)
X = data.drop("target", axis=1)
y = data["target"]

# Step 5: Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 6: Create Logistic Regression model
# Default parameters are used
model = LogisticRegression(max_iter=1000)

# Step 7: Train the model
model.fit(X_train, y_train)
```

```python
# Step 8: Make predictions
y_pred = model.predict(X_test)

# Step 9: Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)

# Step 10: Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Step 11: Display classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output:

```
First 5 records:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   63    1   3       145   233    1        0      150      0      2.3      0
1   37    1   2       130   250    0        1      187      0      3.5      0
2   41    0   1       130   204    0        0      172      0      1.4      2
3   56    1   1       120   236    0        1      178      0      0.8      2
4   57    0   0       120   354    0        1      163      1      0.6      2

   ca  thal  target
0   0     1       1
1   0     2       1
2   0     2       1
3   0     2       1
4   0     2       1

Accuracy: 0.8852459016393442

Confusion Matrix:
[[25  4]
 [ 3 29]]

Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.86      0.88        29
           1       0.88      0.91      0.89        32

    accuracy                           0.89        61
   macro avg       0.89      0.88      0.88        61
weighted avg       0.89      0.89      0.89        61
```

2. Write Python code for binary logistic regression with different parameters using Heart.csv dataset.

```
# Binary Logistic Regression with Different Parameters
# Using Heart.csv Dataset

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
data = pd.read_csv("D:\\A_AMPICS_MCA\\sem_3\\ml\\dataset\\heart.csv")

# Separate features and target
X = data.drop("target", axis=1)
y = data["target"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Logistic Regression with different parameters

# Model 1: L1 Regularization
model1 = LogisticRegression(
    penalty="l1",
    solver="liblinear",
    C=0.1,
    max_iter=1000
)
```

```
# Model 2: L2 Regularization
model2 = LogisticRegression(
    penalty="l2",
    solver="liblinear",
    C=1.0,
    max_iter=1000
)

# Model 3: Higher C value
model3 = LogisticRegression(
    penalty="l2",
    solver="liblinear",
    C=10,
    max_iter=1000
)

# Train models
model1.fit(X_train, y_train)
model2.fit(X_train, y_train)
model3.fit(X_train, y_train)

# Predictions
pred1 = model1.predict(X_test)
pred2 = model2.predict(X_test)
pred3 = model3.predict(X_test)

# Calculate accuracy
accuracy1 = accuracy_score(y_test, pred1)
accuracy2 = accuracy_score(y_test, pred2)
accuracy3 = accuracy_score(y_test, pred3)
```

```
# Display results
print("Binary Logistic Regression Results")
print("-----------------------------------")

print("\nModel 1 (L1, C=0.1)")
print("Accuracy:", accuracy1)
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred1))

print("\nModel 2 (L2, C=1.0)")
print("Accuracy:", accuracy2)
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred2))

print("\nModel 3 (L2, C=10)")
print("Accuracy:", accuracy3)
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred3))

# Classification report for the best model
print("\nClassification Report - Model 2")
print(classification_report(y_test, pred2))
```

Output:

```
Binary Logistic Regression Results
-----------------------------------

Model 1 (L1, C=0.1)
Accuracy: 0.8688524590163934
Confusion Matrix:
[[25  4]
 [ 4 28]]

Model 2 (L2, C=1.0)
Accuracy: 0.8688524590163934
Confusion Matrix:
[[25  4]
 [ 4 28]]

Model 3 (L2, C=10)
Accuracy: 0.8524590163934426
Confusion Matrix:
[[25  4]
 [ 5 27]]

Classification Report - Model 2
              precision    recall  f1-score   support

           0       0.86      0.86      0.86        29
           1       0.88      0.88      0.88        32

    accuracy                           0.87        61
   macro avg       0.87      0.87      0.87        61
weighted avg       0.87      0.87      0.87        61
```

3. Write Python code for multimodal logistic regression with default parameters using multiclass dataset.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
data = pd.read_csv("D:\\A_AMPICS_MCA\\sem_3\\ml\\dataset\\Multiclass Diabetes Dataset\\Multiclass Diabetes Dataset.csv")
# Display first 5 records
print("First 5 records:")
print(data.head())
# Separate input features and target
X = data.drop("Class", axis=1)
y = data["Class"]

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create Logistic Regression model
# Default parameters
model = LogisticRegression(max_iter=1000)

# Train the model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output:

```
First 5 records:
   Gender  AGE  Urea  Cr  HbA1c  Chol   TG  HDL  LDL  VLDL   BMI  Class
0       0   50   4.7  46    4.9   4.2  0.9  2.4  1.4   0.5  24.0      0
1       1   26   4.5  62    4.9   3.7  1.4  1.1  2.1   0.6  23.0      0
2       1   33   7.1  46    4.9   4.9  1.0  0.8  2.0   0.4  21.0      0
3       0   45   2.3  24    4.0   2.9  1.0  1.0  1.5   0.4  21.0      0
4       0   50   2.0  50    4.0   3.6  1.3  0.9  2.1   0.6  24.0      0

Accuracy: 0.9622641509433962

Confusion Matrix:
[[18  0  0]
 [ 1  6  1]
 [ 0  0 27]]

Classification Report:
              precision    recall  f1-score   support

           0       0.95      1.00      0.97        18
           1       1.00      0.75      0.86         8
           2       0.96      1.00      0.98        27

    accuracy                           0.96        53
   macro avg       0.97      0.92      0.94        53
weighted avg       0.96      0.96      0.96        53
```

4. Write Python code for multimodal logistic regression with different parameters using multiclass dataset.

```
# Multiclass Logistic Regression with Different Parameters
# Using Multiclass Diabetes Dataset

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# Load dataset
data = pd.read_csv("D:\\A_AMPICS_MCA\\sem_3\\ml\\dataset\\Multiclass Diabetes Dataset\\Multiclass Diabetes Dataset.csv")

# Separate features and target
X = data.drop("Class", axis=1)
y = data["Class"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model 1: L2 regularization, C=0.1
model1 = LogisticRegression(
    penalty="l2",
    C=0.1,
    max_iter=1000
)

# Model 2: L2 regularization, C=1
model2 = LogisticRegression(
    penalty="l2",
    C=1,
    max_iter=1000
)
```

```
# Model 3: L2 regularization, C=10
model3 = LogisticRegression(
    penalty="l2",
    C=10,
    max_iter=1000
)

# Train models
model1.fit(X_train, y_train)
model2.fit(X_train, y_train)
model3.fit(X_train, y_train)

# Make predictions
pred1 = model1.predict(X_test)
pred2 = model2.predict(X_test)
pred3 = model3.predict(X_test)

# Display accuracy
print("Multiclass Logistic Regression Results")
print("----------------------------------------")

print("\nModel 1 (C = 0.1)")
print("Accuracy:", accuracy_score(y_test, pred1))
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred1))

print("\nModel 2 (C = 1)")
print("Accuracy:", accuracy_score(y_test, pred2))
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred2))

print("\nModel 3 (C = 10)")
print("Accuracy:", accuracy_score(y_test, pred3))
print("Confusion Matrix:")
print(confusion_matrix(y_test, pred3))
```

Output:

```
Multiclass Logistic Regression Results
---------------------------------------

Model 1 (C = 0.1)
Accuracy: 0.9056603773584906
Confusion Matrix:
[[18  0  0]
 [ 2  4  2]
 [ 0  1 26]]

Model 2 (C = 1)
Accuracy: 0.9622641509433962
Confusion Matrix:
[[18  0  0]
 [ 1  6  1]
 [ 0  0 27]]

Model 3 (C = 10)
Accuracy: 0.9245283018867925
Confusion Matrix:
[[18  0  0]
 [ 1  6  1]
 [ 1  1 25]]
```

5. Write python code for Moving average(MA) using Time-Series Analysis Dataset.csv and sales dataset with window value 2,3,4.

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
data = pd.read_csv("D:\\A_AMPICS_MCA\\sem_3\\ml\\dataset\\Sales_data.csv")

# Convert Date column to datetime
data["Date"] = pd.to_datetime(data["Date"])

# Calculate Moving Average with different window values
data["MA_2"] = data["Sales"].rolling(window=2).mean()
data["MA_3"] = data["Sales"].rolling(window=3).mean()
data["MA_4"] = data["Sales"].rolling(window=4).mean()

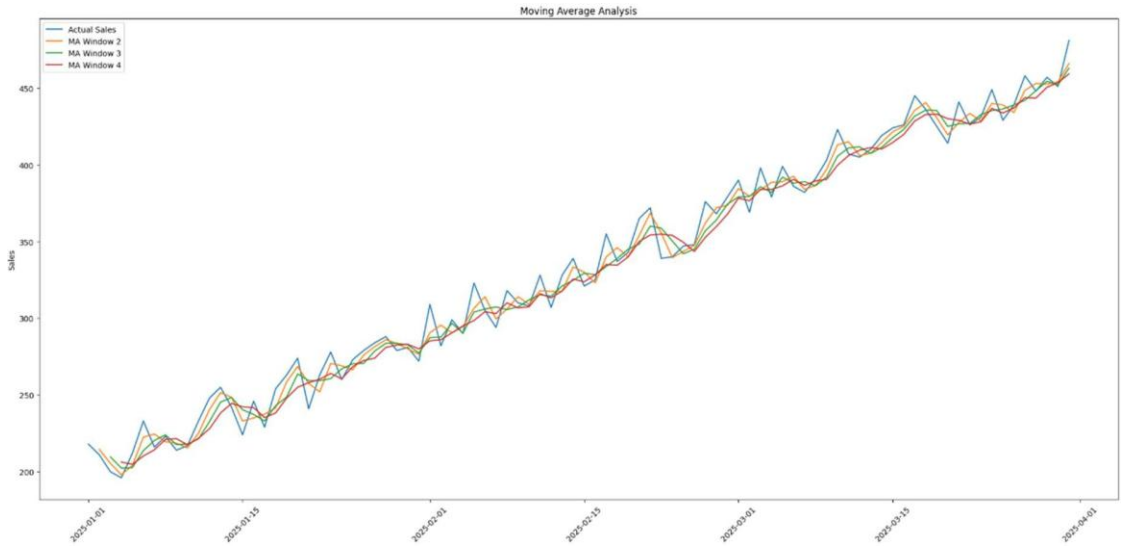
# Display the results
print("Moving Average Results:")
print(data)

# Plot actual sales
plt.figure(figsize=(20, 10))
plt.plot(data["Date"], data["Sales"], label="Actual Sales")

# Plot Moving Averages
plt.plot(data["Date"], data["MA_2"], label="MA Window 2")
plt.plot(data["Date"], data["MA_3"], label="MA Window 3")
plt.plot(data["Date"], data["MA_4"], label="MA Window 4")

plt.xlabel("Date")
plt.ylabel("Sales")
plt.title("Moving Average Analysis")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
Moving Average Results:
         Date  Sales   MA_2        MA_3    MA_4
0  2025-01-01    218    NaN         NaN     NaN
1  2025-01-02    211  214.5         NaN     NaN
2  2025-01-03    200  205.5  209.666667     NaN
3  2025-01-04    196  198.0  202.333333  206.25
4  2025-01-05    212  204.0  202.666667  204.75
..        ...    ...    ...         ...     ...
85 2025-03-27    458  448.5  442.000000  443.75
86 2025-03-28    448  453.0  448.333333  443.50
87 2025-03-29    457  452.5  454.333333  450.50
88 2025-03-30    451  454.0  452.000000  453.50
89 2025-03-31    481  466.0  463.000000  459.25
```



6. Write python code for Exponential Moving Average (EMA)using Time-Series Analysis sales dataset with span value 3,4,5

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the dataset
data = pd.read_csv("D:\\A_AMPICS_MCA\\sem_3\\ml\\dataset\\Sales_data.csv")

# Convert Date column to datetime
data["Date"] = pd.to_datetime(data["Date"])

# Calculate EMA with span values 3, 4, and 5
data["EMA_3"] = data["Sales"].ewm(span=3, adjust=False).mean()
data["EMA_4"] = data["Sales"].ewm(span=4, adjust=False).mean()
data["EMA_5"] = data["Sales"].ewm(span=5, adjust=False).mean()

# Display results
print("Exponential Moving Average Results:")
print(data)

# Simple EMA Graph

plt.figure(figsize=(10, 5))

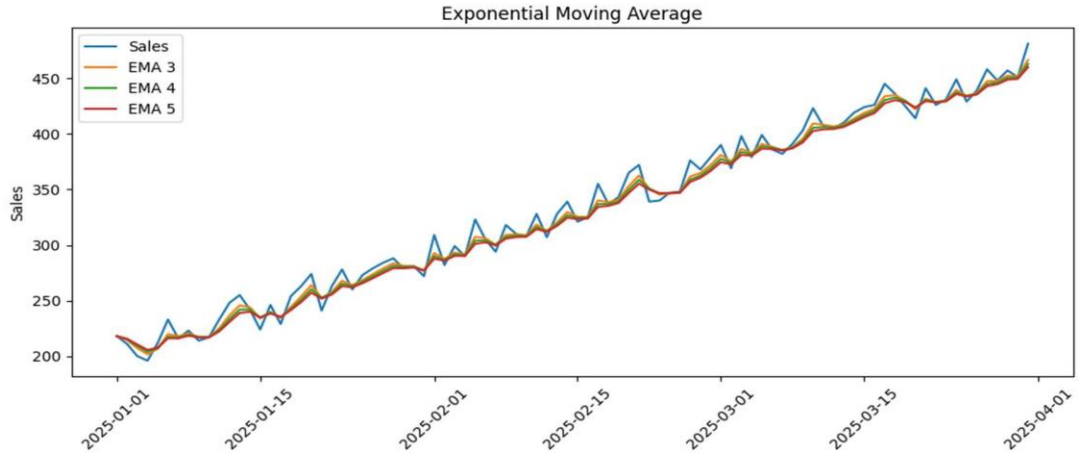
plt.plot(data["Date"], data["Sales"], label="Sales")
plt.plot(data["Date"], data["EMA_3"], label="EMA 3")
plt.plot(data["Date"], data["EMA_4"], label="EMA 4")
plt.plot(data["Date"], data["EMA_5"], label="EMA 5")

plt.xlabel("Date")
plt.ylabel("Sales")
plt.title("Exponential Moving Average")
plt.legend()

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
Exponential Moving Average Results:
         Date  Sales       EMA_3       EMA_4       EMA_5
0  2025-01-01    218  218.000000  218.000000  218.000000
1  2025-01-02    211  214.500000  215.200000  215.666667
2  2025-01-03    200  207.250000  209.120000  210.444444
3  2025-01-04    196  201.625000  203.872000  205.629630
4  2025-01-05    212  206.812500  207.123200  207.753086
..        ...    ...         ...         ...         ...
85 2025-03-27    458  447.304400  444.803530  442.904361
86 2025-03-28    448  447.652200  446.082118  444.602907
87 2025-03-29    457  452.326100  450.449271  448.735271
88 2025-03-30    451  451.663050  450.669562  449.490181
89 2025-03-31    481  466.331525  462.801737  459.993454

[90 rows x 5 columns]
```



7. Write python code for Auto-Regressive Integrated Moving Average(ARIMA)using Time-Series Analysis sales dataset with parameter(2,1,2)

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("D:\\A_AMPICS_MCA\\sem_3\\ml\\dataset\\Sales_data.csv")

df['Date'] = pd.to_datetime(df['Date'])

from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(df['Sales'], order=(2,1,2))

model_fit = model.fit()

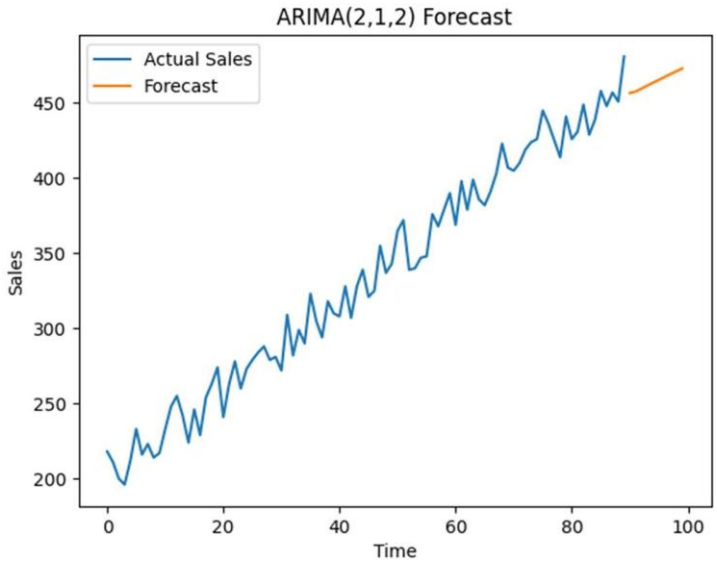
forecast = model_fit.forecast(steps=10)

print("ARIMA Forecast:")
print(forecast)

# Simple graph
plt.plot(df['Sales'], label="Actual Sales")
plt.plot(
    range(len(df), len(df) + 10),
    forecast,
    label="Forecast"
)

plt.title("ARIMA(2,1,2) Forecast")
plt.xlabel("Time")
plt.ylabel("Sales")
plt.legend()
plt.show()
```

```
ARIMA Forecast:
90    456.708931
91    457.667503
92    459.622972
93    461.598521
94    463.555007
95    465.491064
96    467.406841
97    469.302549
98    471.178397
99    473.034594
Name: predicted_mean, dtype: float64
```



8.

```python
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA

# Load dataset
df = pd.read_csv("D:\\A_AMPICS_MCA\\sem_3\\ml\\dataset\\Sales_data.csv")

# Convert Date column to datetime
df["Date"] = pd.to_datetime(df["Date"])

# Create ARIMA model
model = ARIMA(df["Sales"], order=(10, 5, 2))

# Fit the model
model_fit = model.fit()

# Forecast next 10 values
forecast = model_fit.forecast(steps=10)

# Display forecast
print("ARIMA(10,5,2) Forecast:")
print(forecast)

# Simple graph
plt.figure(figsize=(10, 5))

plt.plot(df["Sales"], label="Actual Sales")
plt.plot(
    range(len(df), len(df) + 10),
    forecast,
    label="Forecast"
)

plt.title("ARIMA(10,5,2) Forecast")
plt.xlabel("Time")
plt.ylabel("Sales")
plt.legend()

plt.show()
```

```
ARIMA(10,5,2) Forecast:
90    466.824405
91    484.315828
92    491.389761
93    497.496596
94    508.610782
95    518.137358
96    526.801578
97    544.271571
98    552.190144
99    570.624246
Name: predicted_mean, dtype: float64
```

